Webprogramozás

2. gyakorlat

Eötvös Loránd Tudományegyetem

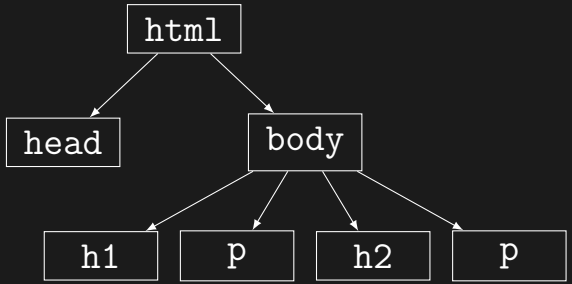
2026-2027/1



DOM - Document Object Model

Document **O**bject **M**odel az oldal HTML kódjának reprezentációja, melyben szerepel minden HTML elemnek megfelelő Javascript objektum.

```
<!DOCTYPE html>
<html>
  <head></head>
  <body>
    <h1>Cím</h1>
    <p>Szöveg</p>
    <h2>Alcím</h2>
    <p>Még több szöveg</p>
  </body>
</html>
```

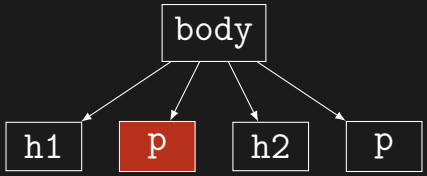


DOM - Elemek elérése

Az elemek kiválasztásához CSS szelektorokat lehet használni.

Vagy az első illeszkedő,

```
document.querySelector('p');
```



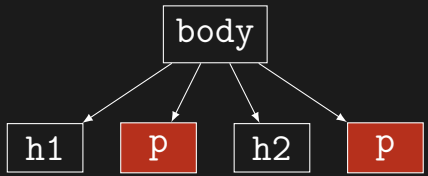
vagy az összes elemet lehet kiválasztani.

```
document.querySelectorAll('p');
```

Utóbbi egy iterálható tömb pl.

```
for (let elem of allElems)
```

segítségével.



DOM - CSS szelektorok

- Elemi szelektorok:
  - Típus: `div`, `span`
  - Osztály: `.classname`
  - Azonosító: `#id`
  - Szűrt típus: `div.classname`, `span#id`
  - Attribútum: `div[attrib=value]`
  - Pszeudo-osztály: `div:hover`, `div:visited`, stb.
  - Pszeudo-elem: `div::before`, `div::after`
- Kombinált szelektorok
  - Leszármazott: `div span`
  - Gyerek: `div > span`
  - Későbbi testvér: `div ~ span`
  - Következő testvér: `div + span`

Forrás: *MDN Web Docs*

DOM - Attribútumok elérése

Legyen a HTML tag-ünk egy

```
<input value="42" data-id="1" readonly
    style="color: blue;">
```

beviteli mező, amelyet le lehet kérdezni és tárolni a

```
const input = document.querySelector('input');
```

segítségével.

Az `input` változón keresztül lehetőség van a tag attribútumait olvasni

```
input.readOnly; // true
input.dataset.id; // "1"
```

és írni is

```
input.value = "1337";
input.style.color = "red";
```

DOM - Tartalom elérése

Legyen a HTML tag-ünk egy

```
<span id="test">Szöveg</span>
```

szöveg, amelyet le lehet kérdezni és tárolni a

```
const span = document.querySelector('#test');
```

segítségével.

A `span` változón keresztül lehetőség van annak belső tartalmának olvasására

```
span.innerHTML; // "Szöveg"
```

és írására is

```
span.innerHTML = "Újabb szöveg";
```

DOM - Elemek hozzáadása és eltávolítása

Egy elem DOM reprezentációja létrehozható a

```
const elem = document.createElement("span");
elem.textContext = "Hello világ!";
```

segítségével.

A létrehozott elem hozzáadható

```
document.body.appendChild(elem);
```

majd el is eltávolítható az

```
elem.remove();
```

segítségével a DOM-ból.

DOM - Feladatok

1. feladat:
Írd ki egymás alá 5x, egyre növekvő betűméretekkel
a "Hello világ!" szöveget!

2. feladat:
Készíts egy táblázat segítségével egy sakktáblát!
(8 x 8-as pálya, bal felső mező fehér)

Események

Az események egy, a felhasználók eset történetekből is ismert **WHEN-THEN** logikát követnek. Amikor (**WHEN**) egy bizonyos esemény bekövetkezik, akkor (**THEN**) egy bizonyos függvény lefut.

```
function handler() { /* THEN */ }
elem.addEventListener("type", handler); // WHEN
```

Események - Delegáció

Legyenek HTML tag-jeink a

```
<div> <input/><input/><input/> </div>
```

beviteli mezők. Tároljuk el azokat a szülőjükkel a

```
const div = document.querySelector("div");
```

segítségével.

Amikor egy eseménykezelőt egy bizonyos elem több, hasonló leszármazottjához kell hozzárendelni, olyankor delegálást használunk. Ekkor az eseménykezelő a szülő elemhez kerül hozzárendelésre.

```
delegate(div, "click", "input", handler);
    // szülő,    WHEN,  gyerek,    THEN
```

Események - Delegáció

A `delegate()` egy segédfüggvény, amely nem része a Javascript beépített könyvtárának, így magunk definiáljuk.

```
function delegate(parent, type, selector, handler) {
  parent.addEventListener(type, function (event) {
    const targetElement = event.target.closest(selector);
    if (this.contains(targetElement))
      handler.call(targetElement, event);
  });
}

        delegate(div, "click", "input", handler);
```

Események - Feladatok

INF-es azonosítóval: `https://progenv.inf.elte.hu`